



Digitalització d'un Taller Mecànic

Connectat

Mòdul: 1665 Digitalització

Nom de l'equip: Guillem Torras i Pep Catalina

Data de lliurament: 26/05/2025

ÍNDEX

ÍNDEX	2
1 INTRODUCCIÓ	3
2 ANÀLISI INICIAL	4
3 MODEL DE DADES	5
4 DESENVOLUPAMENT DE LA SOLUCIÓ	6
4.1 API	6
4.2 WEB	6
4.3 SCRIPTS	8
5 INFRAESTRUCTURA DE SISTEMES	9
6 PROBLEMES I SOLUCIONS	16
7 CRONOGRAMA D'ACTIVITATS	17
8 CONCLUSIONS	18
9 ANNEXOS	19

1 INTRODUCCIÓ

(Breu descripció del projecte: en què consisteix i què es vol aconseguir.)

En aquest projecte de Digitalització (1665), l'objectiu principal és desenvolupar un sistema digital per a la gestió de pacients i cites en un centre mèdic. Per poder aconseguir aquest objectiu, hem creat una solució basada en tres components principals: una pàgina web per a la interacció amb els usuaris, una API per a la comunicació entre serveis i una base de dades per a l'emmagatzematge de la informació. Aquest sistema permet automatitzar tasques administratives bàsiques i millorar l'experiència dels pacients mitjançant una interfície intuïtiva i eficient.

A més, tot el projecte s'ha contenedoritzat amb Docker, la qual cosa facilita el seu desplegament i la seva replicació en diferents entorns de manera ràpida i senzilla.

2 ANÀLISI INICIAL

(Quines necessitats té el client? Quins problemes solucioneu?)

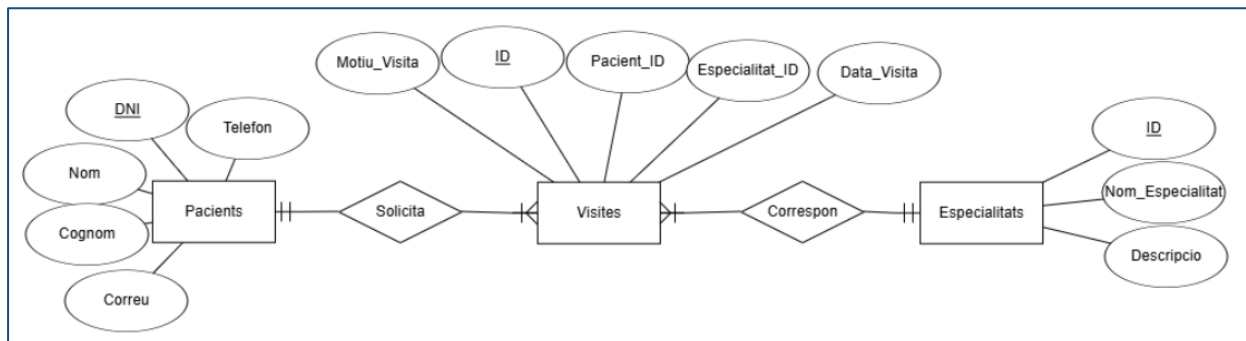
Som un consultori mèdic que hem creat una pàgina web per facilitar la gestió de les cites mèdiques. A través del formulari disponible al web, els pacients poden sol·licitar una cita de manera ràpida i senzilla, sense necessitat de desplaçar-se físicament al centre.

A més, la web també ofereix un apartat on es poden consultar les pròximes visites programades, millorant així l'organització i la comunicació amb els nostres pacients.

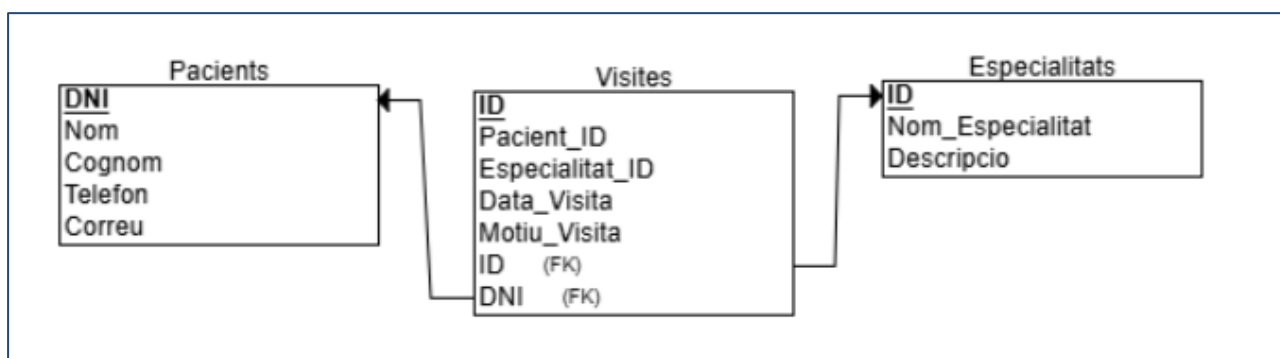
Amb aquesta eina, solucionem un problema habitual: la dificultat per demanar hora de manera presencial, oferint una alternativa còmoda i accessible per a tothom.

3 MODEL DE DADES

ER Diagram:



Relatioal Schema:



Script MySQL DB:



db_schema.sql

4 DESENVOLUPAMENT DE LA SOLUCIÓ

4.1 API

- Llenguatge utilitzat: Python
- Framework: Flask
- Funcionalitats:
 - GET /vehicles
 - POST /appointments
 - GET /appointments

L'API l'hem fet amb un codi base que se'ns va proporcionar, el qual és un codi Python.

Hem utilitzat Flask per poder fer que el formulari envies les dades al servidor de la base de dades, mitjançant una petició POST, i s'insereixin en les taules corresponents. També l'hem utilitzat per poder fer peticions de GET per mostrar les dades en la taula de visites pendents.

L'API funciona enviant una petició POST amb les dades introduïdes en el formulari, el servidor rep les dades i fa INSERTS a les taules de Pacients i Visites.

Per mostrar les dades en la taula, fa una petició GET al servidor de la base de dades, el qual rep una consulta la qual és:

```
SELECT p.dni, p.nom, p.cognom, v.data_visita, v.motiu_visita
FROM pacients p
LEFT JOIN visites v ON p.dni = v.pacient_id
```

4.2 WEB

- HTML/CSS/JS Bàsic
- Incloure captures de pantalles (en format portàtil i mòbil)
- Descriure operativa bàsica per demanar cita

La nostra pàgina web té 3 pàgines: Una pàgina principal, una amb un formulari per demanar visita i l'última per veure les visites que hi ha pendent.

En la pàgina del formulari hem utilitzat el JS per poder validar i enviar les dades del formulari al servidor amb una petició fetch.

En la pàgina on es mostren les visites el JS l'hem utilitzar per mostrar en una taula del HTML els pacients amb les seves visites pendents.

Pàgina en ordinador:

 **Centre Mèdic**

IniciDemandar CitaLlista de Cites

Llista de Cites

DNI	Nom i Cognoms	Data de Cita	Motiu de Consulta
12341234A	Guillem Torras	2025-05-24	Revisió

Pàgina en mòbil:

 **Centre Mèdic**

IniciDemandar CitaLlista de Cites

Llista de Cites

DNI	Nom i Cognoms	Data de Cita	Motiu de Consulta
12341234A	Guillem Torras	2025-05-24	Revisió

Centre Mèdic Privat · Carrer Salut, 123 · Barcelona

✉ info@centremedic.cat | ☎ 933 123 456

Facebook | Instagram | Twitter X

4.3 SCRIPTS

- Script de còpia de seguretat de la base de dades

Aquest script fa una còpia de seguretat de la base de dades de Centre Mèdic a MariaDB.

Primer, posa les dades per connectar-se a la base de dades i la carpeta on es guardarà la còpia.

Després, crea la còpia i la comprimeix en un fitxer amb la data i hora.

Finalment, comprova si la còpia s'ha fet bé i t'ho diu en pantalla amb un missatge en color verd (si tot ha anat bé) o vermell (si hi ha hagut un problema).



backup.sh

- Script de comprovació d'estat dels contenidors

Aquest script comprova si tot funciona bé amb tres serveis diferents:

- 1. Comprova si els contenidors Docker estan en funcionament:**

Revisa si els contenidors anomenats nginx-digitalitzacio, api-digitalitzacio i mariadb-digitalitzacio estan actius.

- 2. Comprova la resposta de l'API:**

Fa peticions a l'API que està en `http://localhost:5000/pacients` fins a 5 vegades, esperant que respongui correctament (codi 200). Si no respon, mostra un error.

- 3. Comprova la connexió a la base de dades MariaDB:**

Intenta connectar-se a la base de dades usant les credencials i fer una consulta senzilla (`SELECT 1;`). Si funciona, confirma que la base de dades està operativa.

Finalment, mostra un resum indicant si tot funciona bé (missatge verd) o si algun servei falla (missatge vermell).



check_services.sh

5 INFRAESTRUCTURA DE SISTEMES

Per poder crear l'estructura necessària hem creat 3 contenidors docker:

- api-digitalitzacio
- mariadb-digitalitzacio
- nginx-digitalitzacio

Api-digitalitzacio:

En aquest contenidor s'allotgen la API que hem fet amb Python. L'únic que es fa en aquest contenidor es copiar-se la nostra API, instal·lar-se els requeriments i a partir d'allà quan reb alguna petició ja s'encarrega de connectar el contenidor de la pagina web i el contenidor de la base de dades.

```
FROM python:3.13-slim
LABEL author="Pep Catalina i Guillem Torres"
LABEL title="Api Digitalitzacio"
LABEL version="1.0"

# Establir directori de treball
WORKDIR /app

# Copiar e instal·lar dependències
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt && pip install net-tools

# Copiar todo el codi de la API
COPY . .

# Exposar el port 5000 de FLASK (5000 per defecto)
EXPOSE 5000

# Iniciar el servidor Flask
CMD ["python", "app.py"]
```

Mariadb-digitalitzacio:

En aquest contenidor es on es crea la base de dades i es guarden las dades, es crea la base de dades juntament amb un parell d'usuaris amb un script que li pesem a l'hora de crear el contenidor. Després es copia el script de Backups de la base de dades fet en bash y es programa per que aquest script s'executi un cop al dia.

```
FROM mariadb:11.3

LABEL author="Pep Catalina i Guillem Torres"
LABEL title="MariaDB con Backup"
LABEL version="1.0"

# Instal·lar cron i utilitats
RUN apt update && \
    apt install -y cron && \
    apt install net-tools && \
    apt clean && \
    mkdir -p /Backup_MySQL

# Copiar scripts i fitxers de configuració
COPY init.sql /docker-entrypoint-initdb.d/init.sql
COPY backup.sh /Backup_MySQL/backup.sh
COPY my-contrab.txt /etc/cron.d/db-backup

# Establir permisos i configurar cron
RUN chmod +x /Backup_MySQL/backup.sh && \
    chmod 0644 /etc/cron.d/db-backup && \
    crontab /etc/cron.d/db-backup

# Iniciar el servidor de la base de dades
CMD ["mariadb"]
```

Nginx-digitalitzacio:

En aquest últim contenidor es on hem muntat la pagina web juntament amb una petita configuració de DNS per poder resoldre per noms en la xarxa que hem crear en docker. Bàsicament s'instal·la els serveix web i DNS i després es copia las configuracions. També es creant uns volums per tal de no perdrà la configuració.

```
FROM alpine:latest

LABEL author="Pep Catalina i Guillem Torres"
LABEL title="NGINX Digitalitzacio"
LABEL version="1.0"

# Instal·lar NGINX i altres utilitats
RUN apk update && apk add --no-cache nginx bind bind-tools iputils net-tools
```

```
# Copiar fitxers de configuració
COPY nginx.conf /etc/nginx/nginx.conf
COPY named.* /etc/bind/
COPY db.* /etc/bind/
COPY centreMedic /var/www/centreMedic

# Creació de volums
VOLUME /var/www
VOLUME /etc/bind

# Exposar ports HTTP i HTTPS
EXPOSE 80 443

# Iniciar el servidor NGINX
CMD ["nginx", "-g", "daemon off;"]
```

El procediment a seguir per poder muntar aquest servidors serà el següent:

```
docker image build -t nginx-digitalitzacio docker/nginx/.
```

```
docker image build -t api-digitalitzacio docker/api/.
```

```
docker image build -t mariadb-digitalitzacio docker/mysql/.
```

```
docker network create --driver bridge --gateway 192.168.1.254 --subnet 192.168.1.0/24 --attachable digi-network
```

```
docker container run -d --name nginx-digitalitzacio --dns 192.168.1.1 --network digi-network --ip 192.168.1.1 -p 8080:80 -p 4433:443 --hostname srv-web nginx-digitalitzacio
```

```
docker container run -d --name api-digitalitzacio --dns 192.168.1.1 --network digi-network --ip 192.168.1.2 -p 5000:5000 --hostname srv-api api-digitalitzacio
```

```
docker container run -d --name mariadb-digitalitzacio -e MYSQL_ROOT_PASSWORD=Educem00 --dns 192.168.1.1 --dns-search centremedic.cat --network digi-network --ip 192.168.1.3 -p 3366:3306 --hostname srv-db mariadb-digitalitzacio
```

Amb el procediment anterior desplegarem els 3 contenidors ja preparats per a funcionar

DOCKER-COMPOSE:

Per simplificar tot aquest procés dels contenidors hem creat un docker-compose.yml per poder desplegar tot el sistema d'una manera mes senzilla i rapida.

En aquest docker-compose s'unifica aquest 3 contenidors creats anteriorment per que amb l'execució d'una sola comanda es desplegui i estigui preparat per a treballar:

```
version: '3.9'

services:
  nginx-digitalitzacio:
    build:
      context: ./docker/nginx
    image: nginx-digitalitzacio
    container_name: nginx-digitalitzacio
    hostname: srv-web
    ports:
      - "8080:80"
      - "4433:443"
    networks:
      digi-network:
        ipv4_address: 192.168.1.1
    dns:
      - 192.168.1.1

  api-digitalitzacio:
    build:
      context: ./docker/api
    image: api-digitalitzacio
    container_name: api-digitalitzacio
    hostname: srv-api
    ports:
      - "5000:5000"
    networks:
      digi-network:
        ipv4_address: 192.168.1.2
    dns:
      - 192.168.1.1

  mariadb-digitalitzacio:
    build:
      context: ./docker/mysql
```

```
image: mariadb-digitalitzacio
container_name: mariadb-digitalitzacio
hostname: srv-db
environment:
  MYSQL_ROOT_PASSWORD: Educem00
ports:
  - "3366:3306"
networks:
  digi-network:
    ipv4_address: 192.168.1.3
dns:
  - 192.168.1.1
dns_search:
  - centremedic.cat

networks:
  digi-network:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.1.0/24
          gateway: 192.168.1.254
```

Per poder desplegar el següent sistema de docker-compose.yaml hem d'executar la següent comanda:

```
docker-compose up -d --build
```

Com es pot veure en la següent captura s'han creat els 3 contenidors i s'han iniciat correctament:

☐	▼	●	projecte_taller	-	-	-	0.03%	1 hour ago		:	
☐		●	api-digitalitza	d1b20f34af25	api-digitalitzac	5000:5000 ↗	0.01%	1 hour ago		:	
☐		●	mariadb-digit	b68b811abb66	mariadb-digita	3366:3306 ↗	0.02%	1 hour ago		:	
☐		●	nginx-digitalit	053279e8c3aa	nginx-digitalitz	4433:443 ↗ Show all ports (2)	0%	1 hour ago		:	

API:

L'API l'hem desenvolupat a partir d'un codi base en Python que se'ns va proporcionar, i hem utilitzat el framework Flask per fer-la funcional.

- **Enviament de dades (POST):**

Quan l'usuari omple i envia el formulari de sol·licitud de cita, es fa una petició HTTP POST des del navegador. Aquesta petició envia les dades al servidor, que rep la informació i fa els inseriments (INSERT) corresponents a les taules de la base de dades: Pacients i Visites.

- **Consulta de dades (GET):**

Per tal de mostrar les cites ja registrades (visites pendents), es fa una petició HTTP GET al servidor. Aquest retorna les dades que després es mostren a la pàgina web.

En resum, l'API permet la **comunicació entre el formulari web i la base de dades**, facilitant tant l'enregistrament com la consulta de dades de manera automàtica i eficient.

JS:

Hem utilitzat JavaScript per millorar la interacció entre l'usuari i el sistema, i per comunicar la pàgina web amb el servidor.

- **A la pàgina del formulari:**

El JavaScript s'encarrega de recollir les dades introduïdes per l'usuari, validar-les i enviar-les al servidor mitjançant una petició fetch amb mètode POST. Això permet enviar les dades sense recarregar la pàgina.

- **A la pàgina de visites pendents:**

El JavaScript fa una petició GET al servidor per obtenir la llista de pacients amb visites programades. Un cop rebudes les dades, es construeix dinàmicament una taula HTML que mostra les cites pendents amb informació com el nom del pacient, la data de la visita i el motiu de consulta.

Comprovació backup BDs:

Per poder fer una comprovació del backup he executat manualment el backup dins del servidor per comprovar que s'executa correctament.

```
root@srv-db:/Backup_MySQL# pwd
/Backup_MySQL
root@srv-db:/Backup_MySQL# ls -l
total 4
-rwxr-xr-x 1 root root 812 May 20 11:23 backup.sh
root@srv-db:/Backup_MySQL# ./backup.sh
Iniciando copia de seguridad de la base de datos 'centro_medico'...
Copia de seguridad creada correctamente: /Backup_MySQL/backup_centro_medico_20250520_112428.sql.gz
root@srv-db:/Backup_MySQL# ls
backup.sh  backup_centro_medico_20250520_112428.sql.gz
root@srv-db:/Backup_MySQL#
```

També he revisat que el cron s'hagi configurat correctament per que es quedi el backup programat per que s'executi cada dia a las 23:53 de la nit.

```
root@srv-db:/Backup_MySQL# cat /etc/cron.d/db-backup
59 23 * * * root /Backup_MySQL/backup.sh >> /var/log/cron.log 2>&1
root@srv-db:/Backup_MySQL#
```

Comprovació de l'estat dels contenidors docker i serveis:

Per poder fer la comprovació dels contenidors i serveis he executat el script hi com es pot observar hem retorna que els 3 contenidors estan iniciats correctament i la api i base de dades responen correctament. Al final de el script fa un resum dient que tot funciona correctament.

```
pepca@PC-Aufe:/mnt/c/Users/pepca/OneDrive - Educem/ASIX1/1665 - Digitalitzacio/projecte_taller/scripts$ ls
backup.sh  check_services.sh
pepca@PC-Aufe:/mnt/c/Users/pepca/OneDrive - Educem/ASIX1/1665 - Digitalitzacio/projecte_taller/scripts$ ./check_services
.sh
Comprovant l'estat dels contenidors...
[OK] El contenedor 'nginx-digitalitzacio' està en execució.
[OK] El contenedor 'api-digitalitzacio' està en execució.
[OK] El contenedor 'mariadb-digitalitzacio' està en execució.
Comprovant la resposta de l'API (http://localhost:5000/pacients)...
[OK] L'API respon correctament.
Comprovant la resposta de la base de dades MariaDB...
[OK] La base de dades respon correctament.
Resum final:
✅ Tot funciona correctament.
pepca@PC-Aufe:/mnt/c/Users/pepca/OneDrive - Educem/ASIX1/1665 - Digitalitzacio/projecte_taller/scripts$
```

6 PROBLEMES I SOLUCIONS

(Quins problemes heu tingut i com els heu resolt.)

Els principals problemes van aparèixer cap al final del projecte, quan vam intentar integrar els tres contenidors (base de dades, API i servidor web). El problema principal era que la nostra API no es comunicava correctament ni amb la base de dades ni amb el servidor web.

Després d'analitzar els errors, vam identificar dues causes principals:

1. Error en la configuració de xarxa:

La API no apuntava a les adreces de xarxa correctes dels contenidors de la base de dades i del servidor web, fet que impedia la comunicació entre ells.

2. Problemes de codificació de caràcters:

Les dades que s'intentaven inserir tenien una codificació diferent de la configurada a la base de dades. Això provocava errors en el moment de guardar-les.

Per solucionar-ho, vam modificar la codificació en las dades que s'inserien a la base de dades perquè utilitzés `utf8mb4_unicode_ci` (Una codificació que si suportava la nostra base de dades), assegurant així la compatibilitat amb caràcters especials.

Un cop feta la modificació, vam comprovar que les dades s'inserien correctament, i vam realitzar diverses proves per verificar que també es guardaven i es mostraven de manera adequada.

7 CRONOGRAMA D'ACTIVITATS

Cronograma d'activitats planificades:

Cronograma d'activitats planificats	Sessió 1	Sessió 2	Sessió 3	Sessió 4	Sessió 5	Sessió 6	Sessió 7	Sessió 8
Guille	Anàlisis Projecte Començar a crear i dissenyar la pagina web	Seguir montant la web amb HTML, CSS i JS	Començar a dissenyar API	Seguir dissenyant API	Seguir dissenyant API			
Pep	Anàlisis Projecte Disseny de l'E/R Creació del Script SQL	Seguir montant la web amb HTML, CSS i JS	Començar a dissenyar imatges Docker: Web, API i DB	Seguir dissenyant imatges Docker: Web, API i DB	Montar Docker Compose	Dockeritzar tot, arreglar possibles errors i comprobar tot el funcionament	Revisio Projecta i fer Documentacio	

Cronograma d'activitats real:

Cronograma d'activitats real	Sessió 1	Sessió 2	Sessió 3	Sessió 4	Sessió 5	Sessió 6	Sessió 7	Sessió 8
Guillem	Anàlisis Projecte Començar a crear i dissenyar la pagina web	Seguir montant la web amb HTML, CSS i JS	Començar a dissenyar API	Seguir dissenyant API	Seguir dissenyant API			
Pep	Anàlisis Projecte Disseny de l'E/R Creació del Script SQL	Seguir montant la web amb HTML, CSS i JS	Començar a dissenyar imatges Docker: Web, API i DB	Seguir dissenyant imatges Docker: Web, API i DB	Montar Docker Compose	Dockeritzar tot, arreglar petits errors i comprobar tot el funcionament	Revisio Projecta i fer Documentacio	

El projecte es va planificar inicialment en un cronograma de 8 sessions, on es van distribuir les diferents tasques a realitzar. Un cop revisat el desenvolupament real, s'ha pogut comprovar que el cronograma s'ha seguit gairebé al peu de la lletra. En algunes sessions ens vam avançar una mica respecte al previst, però en altres vam necessitar una mica més de temps per acabar certes parts, de manera que s'ha compensat de forma equilibrada. Això demostra una bona planificació inicial i una execució coherent amb els objectius marcats.

8 CONCLUSIONS

(Què heu après amb aquest projecte? Què milloraríeu?)

Amb aquest projecte hem après a crear una API que permet connectar una pàgina web amb una base de dades que es troba en un servidor diferent del que allotja la web. Gràcies a aquesta separació hem entès millor com funciona l'arquitectura client-servidor i com pot ajudar a fer que un sistema sigui més flexible i escalable.

L'API fa de pont entre la web i la base de dades, gestionant les peticions a través dels protocols HTTP i HTTPS, i retornant la informació en format JSON. Durant el desenvolupament, hem après a definir les rutes de l'API i a controlar qui pot accedir-hi amb l'ús de capçaleres CORS, assegurant una comunicació segura i correcta.

Hem utilitzat peticions GET per obtenir dades i POST per enviar-ne, especialment des d'un formulari que hem creat a la web. Aquest projecte ens ha ajudat a entendre millor com es connecta el backend amb el frontend i ens ha donat una visió clara de com funcionen les API en projectes reals.

9 ANNEXOS

- Enllaços a repositori de codi

Github: https://github.com/Pep-Catalina/projecte_taller

Dockerhub:

API: <https://hub.docker.com/r/pepcatalina/api-digitalitzacio>

DB: <https://hub.docker.com/r/pepcatalina/mariadb-digitalitzacio>

WEB: <https://hub.docker.com/r/pepcatalina/nginx-digitalitzacio>

- Captures de pantalla del projecte funcionant

Per comprovar que tot està funcionant com toca, començarem creant una visita i veurem si es guarda i es mostra en la taula.

Creem una visita per comprovar que es mostri en la taula i s'afegeixi en la base de dades:

Demana la teva cita

DNI:

Nom i cognoms:

Telèfon:

Ara anem a la pàgina de visites per confirmar si es veu correctament.

I efectivament, s'ha creat la visita correctament:

Llista de Cites			
DNI	Nom i Cognoms	Data de Cita	Motiu de Consulta
12341234A	Guillem Torras	2025-05-24	Revisió
87654321A	Pep Catalina	2025-05-29	Revisió

- Fitxers addicionals



Cronograma
d'activitats.xlsx



1665-Checklist del
Projecte.docx



Presentacio_Digitalitz
acio_Guillem Torras_P